

Intermediate (temperature and cost, side by side)

— Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real formula and mechanism), **Advanced** (the cost and randomness questions a production feature actually has to answer). Read Basic first even if you already feel confident — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and a worked example](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

This has two separate parts, and they don't depend on each other.

Part 1 (temperature): pick a prompt, then imagine (or actually run, once you have Doc04's client) asking it 3 times at `temperature=0` and 3 times at `temperature=1`. Guess first: will the 3 low-temperature answers be nearly identical? Will the 3 high-temperature answers vary a lot, a little, or not at all?

Part 2 (cost): this is just multiplication. You're given (or you look up) a price per input token and a price per output token, and a count of each — multiply and add.

Things to use:

- A genuinely open-ended prompt for Part 1 (not a math question).
- OpenAI's current pricing page for Part 2.
- `2000 × input_price + 500 × output_price` as the formula shape.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Part 1: the key idea to predict *before* you run anything: not every prompt is equally sensitive to temperature. A prompt with essentially one correct answer ("what's 7 times 8?") will barely change even at high temperature, because the model is extremely confident about the next tokens regardless of the sampling setting. A genuinely open-ended prompt ("write a short story about a lighthouse") will visibly change across runs at `temperature=1`, because many different continuations are all reasonably likely.

Part 2: the formula is $(\text{input_tokens} \times \text{input_price_per_token}) + (\text{output_tokens} \times \text{output_price_per_token})$. Prices are usually quoted per 1,000 or per 1,000,000 tokens on a pricing page — convert to a per-token rate first, or keep everything in the same unit (e.g. "per million tokens") throughout the calculation so you don't mix units.

The exact pieces:

- **A good test prompt** — genuinely open-ended but short enough to compare 6 answers side by side quickly, like "give me a one-sentence tagline for a coffee shop."
- **Predict specifically** — will all 3 `temperature=0` answers be word-for-word identical, or just very similar? (In practice: usually very similar, not always byte-identical — there's some randomness elsewhere in the serving system even at `temperature=0`.)
- **Units, made explicit** — if the price is "\$0.15 per 1M tokens," convert it: `0.15 / 1_000_000` dollars per token, or keep the token counts in millions too.

Write your Part 1 prediction and your Part 2 formula (with real numbers plugged in) before checking Hint 2.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Both parts have a hidden production-scale question underneath the one-call version you're computing.

For temperature: `temperature=0` reduces variation but doesn't guarantee an identical answer every single time — there's genuine floating-point non-determinism in how modern GPU serving batches multiple requests together, which slightly changes rounding. This matters for a real feature: if you need *exact* reproducibility (a golden-file test that checks the model's output word-for-word), `temperature=0` alone isn't a strong enough guarantee — you'd need to cache and replay a real response instead of re-calling the model, or accept that "deterministic" here means "extremely consistent," not "provably identical."

For cost: the formula you're using assumes you know `output_tokens` in advance — you don't, until the call finishes. A production feature that wants to alert or cap spend *before* a call completes has to budget for the worst case (the model's `max_tokens` setting) rather than the expected case, since a single unusually long reply can blow past what a typical call costs. There's also a second cost lever this document's Core Concepts only touched on: **prompt caching** — a system prompt resent on every call is often eligible for a steep discount on the *repeated* portion, which changes the real per-call cost far below what `input_tokens × input_price` alone would suggest, but only if that resent portion is genuinely byte-identical every time (the same caveat as the token-count-guessing exercise's trailing-space problem).

The extra pieces:

- **Budgeting for `max_tokens` , not average output length**, when setting a hard spend cap or alert threshold — the average is what you'd use for a monthly cost *estimate*, the max is what you'd use for a *guarantee* that a single call can't cost more than X.
- **Cache-eligible vs. cache-miss cost** — the same call can cost noticeably different amounts depending on whether its repeated prefix (system prompt, few-shot examples) actually hit the cache that call.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both predict variation and compute cost for one call, assuming a known output length and a stable price. Advanced questions both assumptions: `temperature=0` reduces but doesn't guarantee exact reproducibility, and real cost estimation has to account for the worst case (not knowing output length ahead of time) and for prompt caching potentially discounting the repeated portion — neither of which shows up until you're budgeting for a feature that runs thousands of times a day, not a single test call.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and a worked example

Basic Version

```
part 1:
    prompt = "give me a one-sentence tagline for a coffee shop"
    run it 3 times at temperature=0, write down all 3 answers
    run it 3 times at temperature=1, write down all 3 answers
    compare: how different are the 3 low-temp answers from each other?
            how different are the 3 high-temp answers from each other?

part 2:
    input_tokens = 2000
    output_tokens = 500
    look up: input_price_per_token, output_price_per_token
    cost = (input_tokens × input_price_per_token) + (output_tokens × output_price_per_token)
```

A worked cost example, using placeholder rates (check OpenAI's real current pricing page before trusting these numbers for anything real):

```
input:  2000 tokens × $0.15 per 1,000,000 tokens = $0.0003
output: 500 tokens × $0.60 per 1,000,000 tokens = $0.0003
total: about $0.0006 for this one call
```

Do the same calculation with the real, current rates for the model you'd actually use, then write your Part 1 answers, before checking the Intermediate Version below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
part 1:
prompt = "give me a one-sentence tagline for a coffee shop"
for temp in [0, 1]:
    for i in range(3):
        call the model with temperature=temp
        print the answer

compare the 3 temp=0 answers to each other (expect: very close, maybe identical)
compare the 3 temp=1 answers to each other (expect: noticeably different words)

part 2:
cost = (input_tokens / 1_000_000) * input_rate_per_million \
      + (output_tokens / 1_000_000) * output_rate_per_million
```

The same worked example, shown as a small formula you can reuse for any call:

```
input_rate_per_million = 0.15    # dollars – look up the real current rate
output_rate_per_million = 0.60   # dollars

input_tokens = 2000
output_tokens = 500

cost = (input_tokens / 1_000_000) * input_rate_per_million \
      + (output_tokens / 1_000_000) * output_rate_per_million
```

Plug in real current numbers for a model of your choice from OpenAI's pricing page, and compute the cost of this document's own example call (2,000 input / 500 output tokens). Then write out your Part 1 temperature predictions in full sentences — "I expect the temp=0 answers to be ____ because ____" — before checking the Advanced Version below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

part 1 - reproducibility:

run the same prompt at temperature=0, 5 times in a row
were all 5 answers byte-for-byte identical, or "very close but not quite"? write down which one you actually observed.

part 2 - worst-case budgeting:

average_cost_per_call = the Part 2 formula, using typical output length
worst_case_cost_per_call = the same formula, using max_tokens instead
of typical output length
at N calls/day, compare average_cost_per_call × N against
worst_case_cost_per_call × N – how far apart are they?

A worked example for worst-case budgeting:

typical case: 2000 input / 500 output tokens -> \$0.0006 per call (from Hint 1)
worst case: 2000 input / max_tokens=4000 -> roughly 8x the output cost alone

at 10,000 calls/day:

typical: ~\$6/day
worst case if every call hit max_tokens: much higher – a real alert
threshold should be set closer to the worst case, not the typical one,
if the goal is "never silently overspend," not just "estimate roughly
what this normally costs"

Run this arithmetic with your own `max_tokens` setting and call volume, then write one sentence: for a feature you'd actually ship, would you set a spend alert based on the typical cost or the worst-case cost — and why? Compare your answer against the full [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both predict variation for one prompt and compute the cost of one call with known token counts. Advanced tests reproducibility at `temperature=0` directly instead of assuming it, and reframes the cost formula around the question a production feature actually needs answered — not "what does a typical call cost" but "what's the most this could possibly cost, and is my alerting built around that number or the wrong one."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)